



Handling strings

Searching



String comparison

- `int strcmp(char const *s1, char const *s2);`
 - returns a value less than zero if s1 precedes s2 in lexicographical order
 - returns **zero** if s1 and s2 are equal
 - returns a value greater than zero if s1 follows s2

Returns 0 (zero):

This means the strings are exactly equal.

Example: `strcmp("Hello", "Hello")` returns 0.

Returns a negative value (less than zero):

This means s1 comes before s2 in the dictionary.

Example: `strcmp("Apple", "Banana")` returns a negative number.

Returns a positive value (greater than zero):

This means s1 comes after s2 in the dictionary.

Example: `strcmp("Zebra", "Cat")` returns a positive number.

Example: string comparison

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[30] = "Programming Computer ";
    char s2[30] = "Programming";
    if (strcmp(s1, s2))
        printf("string 1 and string 2 are equal");
    else
        printf("string 1 and 2 are different");
    return 0;
}
```

String comparison: first n characters

- `int strncmp(char const *s1, char const *s2, int n);`
 - returns a value less than zero if the first differing character in s1 is smaller than the corresponding character in s2
 - returns zero if first n characters of s1 and s2 are equal
 - returns a value greater than zero if the first differing character in s1 is greater than the corresponding character in s2

String comparison: first n characters

- `int strncmp(const char *str1, const char *str2, size_t n)`
 - Compares first n characters of both the strings

```
#include <stdio.h>
#include <string.h>
int main() {
    char s1[30] = "Programming Computer";
    char s2[30] = "Programming";
    if (strncmp(s1, s2, 11) == 0)
        printf("string 1 and string 2 are equal");
    else
        printf("string 1 and 2 are different");
    return 0;
}
```

Search for character from beginning in string



- `char *strchr(char *str, int ch)`
 - Searches string str for character ch



Search for character from beginning in string

- char ***strchr**(char *str, int ch)
 - Searches string str for character ch

```
#include <stdio.h>
#include <string.h>
int main()
{
    char mystr[40] = "I'm an example of function strchr";
    printf ("%s", strchr(mystr, 'f'));
    return 0;
}
```

Output : from West Bengal

Search for character from end in string



- `char *strchr(char *str, int ch)`
 - Searches string str for character ch in reverse order

Search for character from end in string

- `char *strchr(char *str, int ch)`
 - Searches string `str` for character `ch` in reverse order

```
#include <stdio.h>
#include <string.h>
int main()
{
    char mystr[40] = "I'm an example of function strchr";
    printf ("%s", strchr(mystr, 'f'));
    return 0;
}
```

Search for character from beginning in string



- `char *strchr(char *str, int ch)`
 - Searches string str for character ch



Search for sub string

- `char *strstr(char *str, char *srch_term)`
 - Searchers for string `srch_term` instead of a character in string `str`

```
#include <stdio.h>
#include <string.h>
int main()
{
    char mystr[30] = "I'm an example of strstr function ";
    printf ("%s", strstr(mystr, "str"));
    return 0;
}
```

Declarations and Initializations

```
char s1[] = "beautiful big sky country",
      s2[] = "how now brown cow";
```

Expression	Value	
strlen(s1)	25	
strlen(s2 + 8)	9	count from 'b'
strcmp(s1, s2)	Negative	
printf("%s", s1 + 10)	big sky country	
strcpy(s1+10, s2+8)	beautiful brown cow	
strcat(s2, "!")	how now brown cow!	

Source (s2 + 8): This tells strcpy to start reading from the 9th character of s2 (index 8).
 Destination (s1 + 10): This tells strcpy to start writing at the 11th character of s1 (index 10).

Tokenize

This is a common trick to "chomp" the newline character that fgets reads. `strcspn(str, "\n")` finds the index of the first newline character (`\n`) in `str`. `str[...] = '\0'`; replaces that `\n` with a null terminator, effectively ending the string there.

- `char *strtok(char *str, const char *delims);`
 - splits `str[]` according to given delimiters and returns the next token.
 - It needs to be called in a loop to get all tokens.
 - It returns NULL when there are no more tokens.

used to split a string into smaller pieces (tokens) based on a set of separator characters (delimiters).

```
int main() {  
    char str[500];  
    fgets(str, sizeof(str), stdin);  
    str[strcspn(str, "\n")] = '\0';  
    char *token = strtok(str, " ");
```

This is the FIRST call. It finds the first word in `str`, using a single space (" ") as the delimiter. `token` now points to that word.

```
    while (token != NULL) {  
        for (int i = 0; token[i]; i++)  
            token[i] = tolower(token[i]); //convert to lowercase  
        printf("%s\n", token);  
        token = strtok(NULL, " ");  
    }  
    return 0;  
}
```

This loop will continue as long as `strtok` keeps finding new tokens.

This is the SUBSEQUENT call. `NULL` tells `strtok` to "find the next word in the string you were working on." The loop then repeats with this new token.



Revisiting: Counting words and Chars

- `char *strtok(char *str, const char *delims);`
 - splits `str[]` according to given delimiters and returns the next token.
 - It needs to be called in a loop to get all tokens.
 - It returns `NULL` when there are no more tokens.

```
int main() {
    char str[500];
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';
    char *token = strtok(str, " ");
    while (token != NULL) {
        for (int i = 0; token[i]; i++)
            token[i] = tolower(token[i]); //convert to lowercase
        printf("%s\n", token);
        token = strtok(NULL, " ");
    } return 0;
}
```

Revisiting: Counting words and Chars

Write a function to count the words and characters in a string

```
void CountWordsChars(const char * str, int *words, int *chars )
```

- Use the `char *strtok(char *str, const char *delims);` function
- `strtok()` modifies the string `str`.
- It replaces each delimiter (space, comma, etc.) with a null character (`'\0'`)



Handling strings

Conversion



String conversion uses

You use a conversion function like `atoi()` (ASCII to integer).

```
int age = atoi("25");
```

- Benefits of string conversion
 - Accept user input as strings and process it as numbers or other data types
 - Validate data
 - Error handling

`atoi("123xyz")` might just convert the first part and return 123, silently ignoring the "xyz". This is bad. A more advanced function like `strtol()` (string to long) can tell you where it stopped converting.

String conversion functions

- `double atof(const char *s)`
 - Converts the string `s` to double
- `int atoi(const char *s)`
 - Converts the string `s` to int
- `long atol(const char *s)`
 - Converts the string `s` to long int

Ex: String conversion functions

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    const char *s = "3.14159";
```

```
    double dnumber = atof(s);
```

```
    int inumber = atoi(s);
```

```
    long lnumber = atol(s);
```

```
    printf("The parsed double value is: %f\n", dnumber);
```

```
    printf("The parsed int value is: %d\n", inumber);
```

```
    printf("The parsed long value is: %ld\n", lnumber);
```

```
    return 0;
```

```
}
```

String conversion to double

- `double strtod(const char *s, char **endptr);`
 - Converts the string `s` to double
 - `endptr` is pointer to the first character that could not be converted

Summary



Function	String ->	Header file	Example
atoi()	int	stdlib.h	"123" → 123
atof()	double	stdlib.h	"3.14" → 3.140000
atol()	long	stdlib.h	"12345" → 12345L
strtod()	double	stdlib.h	"123.34a5" → 123.34 (Stops at a)





```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
```

```
    const char *s = "3.14159";
```

```
    char *endptr;
```

```
    double value = strtod(s, &endptr);
```

```
    if (endptr == s)
```

```
        printf("No conversion performed.\n");
```

```
    else {
```

```
        printf("Converted value: %lf\n", value);
```

```
        printf("Characters after conversion: %s\n", endptr);
```

```
    }
```

```
    return 0;
```

```
}
```

strtod converts the entire string.

endptr is set to point to the \0 (null terminator) at the very end of the string.

**endptr is therefore \0.*

Your code prints Characters after conversion: followed by an empty string.

String conversion to long

- `long int strtoll(const char *s, char **endptr, int base);`
 - Converts the string `s` to long int
 - `endptr` is pointer to the first character that could not be converted
 - `base`: base of the number system used in the input string.
- `long int atol(const char *s)`
 - Converts the string `s` to long int



Ex: String conversion to long long

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    const char *s = "12345";
    char *endptr;
    long long int value = strtoll(s, &endptr, 10);
    if (endptr == s) {
        printf("No conversion performed.\n");
    } else {
        printf("Converted value: %lld\n", value);
        printf("Characters after conversion: %s\n", endptr);
    }
    return 0;
}
```



String conversion to unsigned long

- `unsigned long int strtoul(const char *s, char **endptr, int base);`
 - Converts the string `s` to unsigned long
 - `endptr` is pointer to the first character that could not be converted
 - `base`: base of the number system used in the input string.



Handling strings

Arguments to main



Passing Arguments to the main Function

- *Syntax of main function*
 - `int main(void) { . . . }`
 - `int main(int argc, char *argv[ ]) { . . . }`
 - `argc`: count of the number of command line arguments
 - `argv`: array of strings (char pointers) where each string represents one of the command-line arguments

Parameters to main()

- How do these arguments get into the program?
- Information regarding the command line arguments are passed using argc and argv

Command line arguments

- Command line arguments are parameters supplied to a program, when the program is invoked.

```
gcc sum.c -o sum
```

```
./sum 1 2 3
```



Program Name Arguments

Echoing command line arguments

```
#include<stdio.h>
main(int argc, char *argv[])
{
    int i;
    printf("%d\n",argc);
    for(i=0; i<argc; ++i)
        printf("%s\n", argv[i]);
}
```

Echoing command line arguments (2)



- If you run the program with three arguments the values stored will be

```
$/a.out arg1 arg2 arg3  
    argc=4  
    argv[0]=a.out  
    argv[1]= arg1  
    argv[2]= arg2  
    argv[3]= arg3
```

Example code to print arguments

```
int main (int argc, char *argv[]){  
    int i;  
    for (i= 0; i < argc; i++) {  
        printf ("Argv[%d] = %s",i, argv[i]);  
    }  
    return 0;  
}
```

\$./a.out how many

argc = 3

argv[0] = a.out

argv[1] = how

argv[2] = many



Exercise: Command line

Write a C program to compute the sum of numbers given as command line arguments.

Sample output

```
$/sum 2 3
```





Exercise: Compute the sum

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <num1> <num2>\n", argv[0]);
        return 1;
    }
    int a = atoi(argv[1]); // convert string to int
    int b = atoi(argv[2]);
    printf("Sum = %d\n", a + b);
    return 0;
}
```





Exercise: Command line

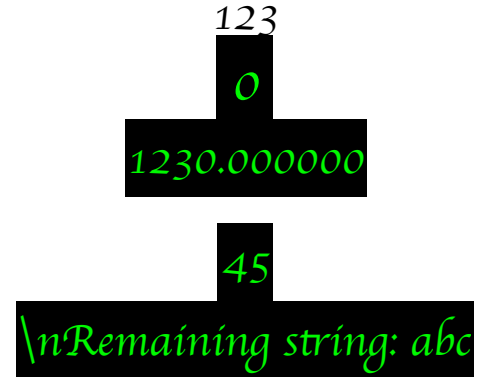
Write a C program to compute the average of numbers given as command line arguments.



Review

What will be the output?

1. `printf("%d", atoi("123abc"));`
2. `printf("%d", atoi("abc123"));`
3. `printf("%f", atof("12.3e2"));`
4. `char *end;`
`printf("%ld", strtol("45abc", &end, 10));`
`printf("\nRemaining string: %s", end);`



123
0
1230.000000
45
\nRemaining string: abc

Exercise



Write a function to count duplicate words in a string.



Exercise



Write a function to count duplicate words in a string.

Step 1: Initialize

```
int main() {  
    char str[500];  
    char *words[100];      // Store the words  
    int wordCount[100] = {0}; // Store the count of each word  
    int count = 0, found;   // Total count of words  
    printf("Enter a string:\n");  
    fgets(str, sizeof(str), stdin);  
    str[strcspn(str, "\n")] = '\0';  
}
```



Exercise



Write a function to count duplicate words in a string.

Step 2: Tokenize

```
char *token = strtok(str, " ");  
while (token != NULL) {  
    // Convert to lowercase for case-insensitive comparison  
    for (int i = 0; token[i]; i++)  
        token[i] = tolower(token[i]);
```



Exercise

Write a function to count duplicate words in a string.

Step 3: Check if word exists

```
found = 0;
for (int i = 0; i < count; i++) {
    if (strcmp(words[i], token) == 0) {
        wordCount[i]++;      found = 1;      break;
    }
}
// If new word, store it
if (!found) {
    words[count] = malloc(20);
    strcpy(words[count], token);
    wordCount[count] = 1;
    count++;
}
token = strtok(NULL, " ,.!?;:-\\t"); }
```

Exercise

Write a function to count duplicate words in a string.

Step 4: Print count of duplicate words

```
printf("\nDuplicate words and their counts:\n");
for (int i = 0; i < count; i++) {
    if (wordCount[i] > 1)
        printf("%s -> %d\n", words[i], wordCount[i]);
}
for(int i = 0; i < count;i++)
    free(words[i]);
return 0;
}
```